

What your retrieval stack is actually worth

An honest audit of the RAG techniques in Iqra Digital Library, benchmarked against the 2026 production standard — plus the three gaps a technical interviewer will find, and how to close them.

Audited apps/api/core/Against 2026 hybrid-retrieval practiceVerdict Strong core, missing evaluation

THE SHORT VERSION

Your retrieval architecture is already at the 2026 production standard. Hybrid BM25 + dense with Reciprocal Rank Fusion at `k=60`, followed by cross-encoder re-ranking, is not a student-project pattern — it is precisely what the current benchmarking literature identifies as the correct default for production systems. You also have two things most portfolio RAG projects lack entirely: HyDE, and a knowledge-graph retrieval channel.

The gap is not technique. It is evidence. There is no evaluation harness anywhere in the repository — no recall@k, no MRR, no NDCG, no golden query set. Every tuning decision in the codebase (the 40/60 BM25-to-FAISS weighting, `k=60` in RRF, fetching `k × 4` candidates before re-ranking) is currently unjustified by measurement. A Senior Manager interview will find that in the first ten minutes, because “*how do you know it’s better?*” is the question that separates someone who assembled a pipeline from someone who owns one.

Close that one gap and this project argues for you at the level you are targeting.

01 — What you built, named correctly

Use these names. Interviewers grade partly on whether you know the vocabulary for what you did, and several of these are described loosely in your current README.

IN YOUR CODE	THE TECHNIQUE’S REAL NAME	WHY IT MATTERS
hybrid_retriever.py <code>40 / 60</code>	Sparse–dense hybrid retrieval with min-max score fusion	BM25 catches exact terms, rare names, and identifiers that embeddings systematically miss; dense catches paraphrase and synonymy. Min-max normalising each score <i>per query</i> before the weighted sum is the correct detail — raw BM25 and cosine live on incomparable scales
_rrf_merge() <code>k=60</code>	Reciprocal Rank Fusion	Rank-based rather than score-based fusion, so it needs no calibration between retrievers. <code>k=60</code> is the canonical constant and remains the 2026 default
generate_hypothetical_doc()	HyDE — Hypothetical Document Embeddings	Closes the query–document asymmetry: a five-word query and a 200-word book description occupy different regions of embedding space. Generating an ideal fake document and embedding <i>that</i> puts the query vector in document space

IN YOUR CODE	THE TECHNIQUE'S REAL NAME	WHY IT MATTERS
<code>generate_query_variants()</code>	Multi-query expansion with rank fusion	Hedges against a single bad phrasing. Books surfacing across several variants get promoted by RRF — a cheap consensus signal
<code>_get_reranker()</code> <code>ms-marco-MiniLM-L-6-v2</code>	Cross-encoder re-ranking (late fusion)	The highest-precision component you have. A bi-encoder scores query and document independently; a cross-encoder sees the pair jointly through full attention. Anthropic's published benchmarks put re-ranking as the step that takes contextual hybrid retrieval from a 2.9% to a 1.9% failure rate
<code>_build_graph()</code> <code>NETWORKX</code>	Graph-augmented retrieval (author / category edges)	A third retrieval channel most portfolio projects don't have. Your live index builds 6,810 nodes and 69,116 edges, traversing shared-author and shared-category links to surface candidates neither lexical nor vector search would reach
<code>agent.py</code> + <code>agents/</code>	Agentic RAG — hierarchical ReAct orchestration	An orchestrator delegating to a SearchWorker and a CuratorWorker is a supervisor pattern, not a single-agent tool loop. Retrieval becomes a decision the model makes rather than a fixed pre-step
<code>llm_provider.py</code>	Provider abstraction with graceful degradation	Five backends behind one interface, and — more importantly — every RAG stage falls back cleanly when the LLM is off or a package is missing. That is an operational instinct, and it is rarer in candidates than any single technique here

The one-sentence version to have ready: “It’s a three-channel hybrid retriever — BM25, dense vectors, and a knowledge graph — fused with Reciprocal Rank Fusion, optionally expanded with HyDE and multi-query, then re-ranked with a cross-encoder, behind a LangGraph supervisor agent that decides when to retrieve at all.”

02 — Scorecard vs the 2026 standard

	DIMENSION	YOUR IMPLEMENTATION	AGAINST THE STANDARD
A	Fusion strategy	RRF at k=60, plus weighted min-max hybrid	Matches the documented production default exactly
A	Re-ranking	Cross-encoder, lazy-loaded, degrades to hybrid order on failure	At standard; the degradation path is above it
A	Query expansion	HyDE + multi-query, independently toggleable	Above standard — most systems ship one, not both
B	Retrieval channels	Sparse + dense + graph	Above standard on breadth; the sparse channel could learn (see SPLADE)
B	Agentic layer	LangGraph supervisor, two workers, MemorySaver threads	At standard. Missing: self-correction on poor retrieval
C	Embedding model	<code>all-MiniLM-L6-v2</code> , 384-dim	Below. Fast and free, but a 2021 model — it sets the ceiling for everything downstream
D	Evaluation	None	Well below. This is the gap that matters
D	Observability	Logging only — no latency or quality metrics retained	Below, though the new v3 pipeline trace is the first half of the fix

Read that honestly: six of eight dimensions sit at or above the current production bar. Both failing grades are about *measurement*, not retrieval technique — and that is a far easier thing to fix than an architecture problem.

03 — The three real gaps

G1 No evaluation harness

Highest priority

Nothing in the repository measures retrieval quality. That means every constant in the system is currently a guess:

- `0.40 / 0.60` BM25-to-FAISS weighting in `hybrid_retriever.py`
- `_RRF_K = 60` — correct by convention, but unverified *on your corpus*
- `fetch_k = k * 4` before re-ranking
- Three query variants rather than five

Each is defensible. None is currently defended. Build a golden set of 50–100 queries with known-relevant books, then report **recall@10**, **MRR**, and **NDCG@10** for each configuration: BM25 alone, dense alone, hybrid, hybrid + HyDE, hybrid + re-rank, and everything on.

The output is an ablation table. That single table changes the conversation from “I implemented these techniques” to “re-ranking bought 14 points of NDCG and cost 180 ms; HyDE bought 3 points and cost 900 ms, so it ships off by default.” The second sentence is a Senior Manager answer. The first is not.

EFFORT One to two days, and the single best return in this document.

G2 Dated embedding model

Medium priority

`all-MiniLM-L6-v2` is 384-dimensional and from 2021. Everything downstream — fusion, re-ranking, the graph — is bounded by what this model retrieves in the first place.

Swap in `BAAI/bge-base-en-v1.5` or `intfloat/e5-base-v2` (both 768-dim, both free, both CPU-viable). Keep MiniLM behind a config flag and *report both in the ablation table* — “I upgraded the encoder and measured the delta” is a stronger story than having quietly used the better model from the start.

Deployment caveat: 768 dimensions doubles index size and cold-start embedding time. Your measured cold build is already ~120 s locally; on the Hugging Face free tier, measure before committing. This may end up local-only with MiniLM in production — which is itself a legitimate trade-off to talk about.

EFFORT Half a day, plus a full index rebuild.

The pipeline is strictly feed-forward: retrieve, fuse, re-rank, answer. It never asks whether the retrieval was any good. Corrective RAG grades retrieved candidates for relevance and, when confidence is low, falls back to a different strategy — a rewritten query, or the external Google Books channel you already have.

You are unusually well positioned for this: the cross-encoder already produces a calibrated relevance score. If the top score falls below a threshold, re-route. That is roughly thirty lines in `rag_pipeline.py`, and it upgrades the description from “agentic RAG” to “agentic RAG with a corrective fallback.”

EFFORT Half a day. Do it after G1 — you need the harness to prove it helps.

04 — What *not* to add

Knowing what to leave out is itself an interview signal. Three techniques you will be tempted toward that are wrong for this corpus:

CONTEXTUAL RETRIEVAL

Anthropic’s method cuts retrieval failures 49% by prepending LLM-generated document context to each chunk before indexing. It is the strongest recent result in RAG — and it does not apply here. It solves *chunks losing the context of their parent document*. Your corpus is ~6,800 atomic book records; there is no parent document and no chunking. Say exactly this if asked. Recognising when a celebrated technique doesn’t fit is worth more than bolting it on.

CHUNKING STRATEGIES

Semantic chunking, sliding windows, small-to-big — all irrelevant for the same reason. A title plus a short description is already the right retrieval unit. Do not invent a chunking problem in order to demonstrate a chunking solution.

A MANAGED VECTOR DATABASE

Pinecone, Weaviate, and Qdrant earn their keep at millions of vectors with concurrent writes. At 6,810 records, FAISS `IndexFlatIP` does *exact* search in single-digit milliseconds — your live API returns a full hybrid query in 71 ms. Migrating would add cost and a network hop to buy nothing.

The one genuine upgrade to your sparse channel

If you want a fourth retrieval channel with real benchmark support, it is **SPLADE** — a learned sparse retriever that expands documents with semantically related terms while keeping an invertible, BM25-like index. Three-way hybrid (dense + BM25 + SPLADE) with a late-interaction re-ranker is the strongest documented configuration in current blended-retrieval benchmarks. It is real work, and it belongs after G1, not before.

And the one worth knowing about but not building

ColBERT and late-interaction retrieval sit between bi-encoders and cross-encoders: per-token embeddings scored by MaxSim, giving near cross-encoder accuracy at near bi-encoder speed. Worth being able to describe — “I’d reach for late interaction if my re-ranking latency budget got tight at scale” — but at 6,810 documents your cross-encoder is already affordable, so adopting it would be solving a problem you don’t have.

05 — Interview answers

The questions this project invites, and answers that hold up under follow-up.

❓ Why hybrid search instead of pure vector search?

Because they fail differently. Dense retrieval handles paraphrase and synonymy but is unreliable on exact tokens — a specific author name, an ISBN, an unusual title. BM25 is the opposite. The benchmarking work through 2025 and 2026 consistently finds BM25 beating strong commercial embedding models on most metrics, which surprises people; the practical conclusion isn’t to pick one, it’s to fuse them. I weight 40% BM25 to 60% dense, min-max normalised per query so the scales are comparable.

If pressed on the weights: be honest — “that was a starting point; an ablation harness is how I’d tune it, and it’s the next thing I’d build.” Admitting an unmeasured constant lands far better than defending it.

❓ Why RRF rather than just averaging the scores?

Because scores from different retrievers aren’t comparable, and their distributions shift per query. RRF ignores magnitudes and uses rank position — one over rank plus 60 — so it needs no calibration between systems. The constant dampens differences at the top of each list, which stops one retriever’s confident-but-wrong top hit from dominating the merge. I use both methods: weighted min-max fusion inside the hybrid retriever where I control both scorers, and RRF across query variants where I don’t.

❓ What does the cross-encoder actually do that the embeddings don’t?

Bi-encoders embed the query and the document separately, so the two never interact until a dot product at the very end — that’s what makes them cheap enough to index a whole corpus. A cross-encoder concatenates the pair and runs full attention across both, so it models term-level interaction directly. Much more accurate, far too slow to run over everything — so it only ever sees the top candidates from the cheap stage. Retrieve broad and cheap, then re-rank narrow and expensive.

❏ Why HyDE? Isn't generating a fake document strange?

It addresses a real asymmetry. A user types five words; the indexed documents are a couple of hundred. Those occupy different regions of embedding space, so cosine similarity between them is measuring across a gap. HyDE has the LLM write what an ideal matching document would look like and embeds *that* instead, so I'm comparing document to document. The factual accuracy of the generated text doesn't matter — it never reaches the user. It only has to land in the right neighbourhood.

The mature addition: it costs an LLM round trip per search, so it's off by default and user-toggable.

❏ How do you know any of this actually improved results?

This is the one to prepare for hardest, and the honest answer today is that I have architectural justification but not measurement — which is exactly why an ablation harness is the next thing I'd build. **Build G1 first and you answer this with numbers instead.**

Until then, state plainly what is designed and what is measured; a candidate who distinguishes the two is trusted more than one who blurs them.

❏ Why keep Python for retrieval instead of rewriting everything in TypeScript?

Because FAISS, the sentence-transformers cross-encoder, and LangGraph have no equivalent in the JavaScript ecosystem. Rewriting would have meant swapping exact vector search for an approximate JS library, dropping cross-encoder re-ranking entirely, and rebuilding the agent with fewer capabilities — losing the parts that make the system interesting, in order to buy a uniform language. So I split it: Python owns retrieval and reasoning, TypeScript owns the interface, and a typed OpenAPI contract joins them so the two can't drift. That's also how production AI systems are actually organised.

❏ What breaks if the corpus goes from 7,000 books to 7 million?

Four things change. FAISS `IndexFlatIP` is exact brute force — fine in memory at this size, untenable at millions, so I'd move to an approximate index, IVF-PQ or HNSW, and accept a recall trade-off I'd measure. The in-memory BM25 would move to a real inverted index. The NetworkX graph is $O(V+E)$ in RAM — mine is already 69,000 edges at 6,810 books — so it needs a graph store or gets dropped. And the cross-encoder stops being free; I'd cap the candidate set and batch on GPU. The architecture survives. Three of the four implementations don't.

❓ Where does this break in production?

Cold start is the honest answer. Loading the encoder, building the FAISS index, fitting BM25, and constructing the graph takes about two minutes from cold locally, which is why the v3 build bakes those artifacts into the container image rather than computing them at boot. Beyond that: the cross-encoder is the latency floor on any re-ranked query; external book APIs fail independently and are wrapped so an outage degrades the result set rather than failing the request; and the public LLM endpoints are rate-limited per IP because the keys are mine.